

LIST DataStructure

```
In [1]: l = []    # Empty List  
l
```

```
Out[1]: []
```

```
In [2]: type(l)
```

```
Out[2]: list
```

```
In [3]: len(l)
```

```
Out[3]: 0
```

```
In [4]: l.append(10)  # Put cursor on 'append' and click shift + tab to get the info of t  
l
```

```
Out[4]: [10]
```

```
In [5]: len(l)
```

```
Out[5]: 1
```

```
In [6]: l.append(20, 30, 40)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[6], line 1  
----> 1 l.append(20, 30, 40)  
  
TypeError: list.append() takes exactly one argument (3 given)
```

```
In [7]: l
```

```
Out[7]: [10]
```

```
In [8]: l.append(20)    # Click tab after writing l. --- can see the all lists of the fun  
l.append(30)  
l.append(40)  
l.append(50)  
l
```

```
Out[8]: [10, 20, 30, 40, 50]
```

```
In [9]: len(l)
```

```
Out[9]: 5
```

```
In [10]: l
```

```
Out[10]: [10, 20, 30, 40, 50]
```

```
In [11]: l.append(10)    # If you run the same code again and again then the output will a
1
```

```
Out[11]: [10, 20, 30, 40, 50, 10]
```

```
In [12]: l1 = [10, 3.5, True, 1 + 2j, 'Welcome']
l1
```

```
Out[12]: [10, 3.5, True, (1+2j), 'Welcome']
```

```
In [13]: l1.append([1,2,3])    # Nested list (List inside List)
l1
```

```
Out[13]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [14]: print(l)
print(l1)
```

```
[10, 20, 30, 40, 50, 10]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [15]: l
```

```
Out[15]: [10, 20, 30, 40, 50, 10]
```

```
In [16]: l3
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 l3

NameError: name 'l3' is not defined
```

```
In [17]: l3 = l.copy()    # Copies the element of list to another list
```

```
In [18]: l3
```

```
Out[18]: [10, 20, 30, 40, 50, 10]
```

```
In [19]: print(l)
print(l1)
print(l3)
```

```
[10, 20, 30, 40, 50, 10]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
[10, 20, 30, 40, 50, 10]
```

```
In [20]: l == l3
```

```
Out[20]: True
```

```
In [21]: len(l) == len(l3)
```

```
Out[21]: True
```

```
In [22]: id(l) == id(l3)
```

Out[22]: False

In [23]: `id(l) != id(l3)`

Out[23]: True

In [24]: `print(id(l))`
`print(id(l1))`
`print(id(l3))`

2176151813440
2176152368960
2176152469248

In [25]: `print(id(l))`
`print(id(l3))`

2176151813440
2176152469248

In [26]: `print(l)`
`print(l1)`
`print(l3)`

[10, 20, 30, 40, 50, 10]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
[10, 20, 30, 40, 50, 10]

In [27]: `l3.clear()`

In [28]: `print(l)`
`print(l1)`
`print(l3)`

[10, 20, 30, 40, 50, 10]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
[]

In [29]: `del l3`

In [30]: `l3`

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[30], line 1  
----> 1 l3  
NameError: name 'l3' is not defined
```

In [31]: `print(l)`
`print(l1)`

[10, 20, 30, 40, 50, 10]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

In [32]: `l`

Out[32]: [10, 20, 30, 40, 50, 10]

In [33]: `l.count(20)`

Out[33]: 1

```
In [34]: l.count(30)
```

Out[34]: 1

```
In [35]: l.count(10)
```

Out[35]: 2

```
In [36]: l
```

Out[36]: [10, 20, 30, 40, 50, 10]

```
In [37]: print(l)
         print(l1)
```

[10, 20, 30, 40, 50, 10]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

```
In [38]: l.extend(l1)
         l
```

Out[38]: [10, 20, 30, 40, 50, 10, 10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

```
In [39]: print(len(l))
         print(len(l1))
```

12
6

```
In [40]: a = [1, 2, 3]
         b = [4, 5, 6]
```

```
In [41]: a.extend(b)
         a
```

Out[41]: [1, 2, 3, 4, 5, 6]

```
In [42]: print(a)
         print(b)
```

[1, 2, 3, 4, 5, 6]
[4, 5, 6]

```
In [43]: print(l)
         print(l1)
```

[10, 20, 30, 40, 50, 10, 10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
[10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

```
In [44]: l.index(20)
```

Out[44]: 1

```
In [45]: l1
```

Out[45]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

```
In [46]: l1.index(1 + 2j)
```

```
Out[46]: 3
```

```
In [47]: l1.index('Welcome')
```

```
Out[47]: 4
```

```
In [48]: l1
```

```
Out[48]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [49]: l1[:]
```

```
Out[49]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [50]: l1[6]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[50], line 1  
----> 1 l1[6]  
  
IndexError: list index out of range
```

```
In [51]: l1[4]
```

```
Out[51]: 'Welcome'
```

```
In [52]: l1
```

```
Out[52]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [53]: l1[:3]
```

```
Out[53]: [10, 3.5, True]
```

```
In [54]: l1
```

```
Out[54]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [55]: l1[3:]
```

```
Out[55]: [(1+2j), 'Welcome', [1, 2, 3]]
```

```
In [56]: l1[::3]
```

```
Out[56]: [10, (1+2j)]
```

```
In [57]: l1
```

```
Out[57]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [58]: l1[0:5:2]
```

```
Out[58]: [10, True, 'Welcome']
```

```
In [59]: l1[0:6:2]
```

```
Out[59]: [10, True, 'Welcome']
```

```
In [60]: l1
```

```
Out[60]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [61]: l1[-3]
```

```
Out[61]: (1+2j)
```

```
In [62]: l1[-3:]
```

```
Out[62]: [(1+2j), 'Welcome', [1, 2, 3]]
```

```
In [63]: l1[::-1] # reverse the list
```

```
Out[63]: [[1, 2, 3], 'Welcome', (1+2j), True, 3.5, 10]
```

```
In [64]: l1
```

```
Out[64]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [65]: l1[::-2]
```

```
Out[65]: [[1, 2, 3], (1+2j), 3.5]
```

```
In [66]: l1
```

```
Out[66]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [67]: l1[4]
```

```
Out[67]: 'Welcome'
```

```
In [68]: print(l1[4][0])  
print(l1[4][1])  
print(l1[4][2])  
print(l1[4][3])  
print(l1[4][4])  
print(l1[4][5])  
print(l1[4][6])
```

W
e
l
c
o
m
e

```
In [69]: print(l1[4][0])  
print(l1[4][1])  
print(l1[4][2])  
print(l1[4][3])  
print(l1[4][4])
```

```
print(l1[4][5])
print(l1[4][6])
print(l1[4][7])
```

W
e
l
c
o
m
e

```
-----
IndexError                                Traceback (most recent call last)
Cell In[69], line 8
      4 print(l1[4][3])
      5 print(l1[4][4])
      6 print(l1[4][5])
      7 print(l1[4][6])
----> 8 print(l1[4][7])

IndexError: string index out of range
```

In [70]: l1

Out[70]: [10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

In [71]: l1[0]

Out[71]: 10

In [72]: l1[0] = 100 *# List is mutable/ changeable*

In [73]: l1

Out[73]: [100, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

In [74]: print(l)
print(l1)

[10, 20, 30, 40, 50, 10, 10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
[100, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

In [75]: l2 = [10,20,30]
l2[0]

Out[75]: 10

In [76]: l2[0] = 100
l2

Out[76]: [100, 20, 30]

In [77]: l2.insert(2,25) *# Adds an element to the assigned index*

In [78]: l2

Out[78]: [100, 20, 25, 30]

```
In [79]: l2.insert(2,[1,2,3])  
l2
```

```
Out[79]: [100, 20, [1, 2, 3], 25, 30]
```

```
In [80]: l2.pop()    # Remove the last element by default
```

```
Out[80]: 30
```

```
In [81]: l2
```

```
Out[81]: [100, 20, [1, 2, 3], 25]
```

```
In [82]: l
```

```
Out[82]: [10, 20, 30, 40, 50, 10, 10, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]
```

```
In [83]: l.pop()
```

```
Out[83]: [1, 2, 3]
```

```
In [84]: l
```

```
Out[84]: [10, 20, 30, 40, 50, 10, 10, 3.5, True, (1+2j), 'Welcome']
```

```
In [85]: l.pop(-2)    # Removes the element according to the assigned index
```

```
Out[85]: (1+2j)
```

```
In [86]: l
```

```
Out[86]: [10, 20, 30, 40, 50, 10, 10, 3.5, True, 'Welcome']
```

```
In [87]: l.remove(True)  
l
```

```
Out[87]: [10, 20, 30, 40, 50, 10, 10, 3.5, 'Welcome']
```

```
In [88]: print(l)  
print(l1)  
print(l2)
```

```
[10, 20, 30, 40, 50, 10, 10, 3.5, 'Welcome']  
[100, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]  
[100, 20, [1, 2, 3], 25]
```

```
In [89]: l
```

```
Out[89]: [10, 20, 30, 40, 50, 10, 10, 3.5, 'Welcome']
```

```
In [90]: l.reverse()    # Reverse the elements of the list  
l
```

```
Out[90]: ['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10]
```

```
In [91]: l[:: -1]
1
```

```
Out[91]: ['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10]
```

```
In [92]: l3 = ['a', 'b', 'c', 'd']
```

```
In [93]: l3
```

```
Out[93]: ['a', 'b', 'c', 'd']
```

```
In [94]: l3.reverse()
l3
```

```
Out[94]: ['d', 'c', 'b', 'a']
```

```
In [95]: l3.sort()    # By default - Ascending Order
l3
```

```
Out[95]: ['a', 'b', 'c', 'd']
```

```
In [96]: l
```

```
Out[96]: ['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10]
```

```
In [97]: l.sort()
l
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[97], line 1
----> 1 l.sort()
      2 l

TypeError: '<' not supported between instances of 'float' and 'str'
```

```
In [98]: l5 = [3, 200, 23, 1000, 10]
l5
```

```
Out[98]: [3, 200, 23, 1000, 10]
```

```
In [99]: l5.sort()
l5
```

```
Out[99]: [3, 10, 23, 200, 1000]
```

```
In [100... l5.sort(reverse=False)    # Ascending Order due to the "False" statement
l5
```

```
Out[100... [3, 10, 23, 200, 1000]
```

```
In [101... l5.sort(reverse=True)    # Descending Order due to the "True" statement
l5
```

```
Out[101... [1000, 200, 23, 10, 3]
```

In [102...]

1

Out[102...]

['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10]

In [103...]

l1

Out[103...]

[100, 3.5, True, (1+2j), 'Welcome', [1, 2, 3]]

In [104...]

```
l1.sort()    # Applies to the same datatype
l1
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[104], line 1
----> 1 l1.sort()    # Applies to the same datatype
      2 l1

TypeError: '<' not supported between instances of 'complex' and 'bool'
```

In [105...]

1

Out[105...]

['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10]

In [106...]

```
10 in l    # List membership (10 is present in list)
```

Out[106...]

True

In [107...]

```
100 in l
```

Out[107...]

False

In [108...]

```
for i in l:
    print(i)
```

```
Welcome
3.5
10
10
50
40
30
20
10
```

In [109...]

```
for i in enumerate(l):    # Shows both index and values which are stored in it
    print(i)
```

```
(0, 'Welcome')
(1, 3.5)
(2, 10)
(3, 10)
(4, 50)
(5, 40)
(6, 30)
(7, 20)
(8, 10)
```

In [110...]

1

Out[110...] ['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10]

In [111...] `all(1)`

Out[111...] True

In [112...] `any(1)`

Out[112...] True

In [113...] `l.append(0)`
1

Out[113...] ['Welcome', 3.5, 10, 10, 50, 40, 30, 20, 10, 0]

In [114...] `all(1)` *# Shows "False" as the List has "0" element*

Out[114...] False

In [115...] `any(1)` *# Shows "True" as the List have other items having the True value*

Out[115...] True

In []: